

BTS SIO – Option SLAM

DOCUMENTATION TECHNIQUE

Projet MediaStock

Architecture • Base de données • API PHP • Sécurité • Déploiement Docker

Auteur	Étudiant BTS SIO SLAM
Projet	MediaStock – Projet n°2
Version	1.0 – Version finale
Date	Mai 2026

1. Architecture Générale

1.1. Vue d'Ensemble

MediaStock repose sur une architecture 3-tiers conteneurisée (Docker), exposée en HTTPS par Traefik :

Couche	Composants	Rôle
Présentation	HTML5 + Bootstrap 5.3 + JS ES6 + html5-qrcode.js	Interface utilisateur responsive, scan QR Code
Application	PHP 8.2 procédural + PDO	Traitement requêtes, logique métier, exposition API JSON
Données	MySQL 8.x (conteneur db)	Persistance : Items, Prêts, Emprunteurs, Formations, Catégories
Infrastructure	Docker Compose + Traefik	3 conteneurs orchestrés, HTTPS port 4433

1.2. Flux de Communication

- Navigateur → Traefik (HTTPS:4433) → Conteneur web (Apache/PHP)
- JS Frontend → fetch() JSON → API PHP (/public/api/*.php) → PDO → MySQL
- Scanner QR Code → html5-qrcode.js → findbyqrcode.php → réponse JSON → formulaire prêt/retour



URL de production : <https://mediastock.iris.a3n.fr:4433> — Le port 4433 est exposé par Traefik avec TLS.

2. Infrastructure Docker

2.1. Services Docker Compose

Service	Image	Rôle	Réseau
web	php:8.2-apache	Serveur Apache + PHP 8.2, expose l'application	mediastock_net
db	mysql:8	MySQL 8, données persistées via volume Docker	mediastock_net
phpmyadmin	phpmyadmin/phpmyadmin	Interface admin BDD, routé via Traefik	mediastock_net

2.4. Sécurité Traefik

- Redirection HTTP → HTTPS automatique sur le port 4433
- Certificat TLS géré par Traefik (Let's Encrypt ou auto-signé selon environnement)
- MySQL non exposé à l'extérieur — réseau Docker interne uniquement
- phpMyAdmin : PMA_ARBITRARY=1 → authentification manuelle obligatoire via l'url : <https://pma-mediastock.iris.a3n.fr:4433>

3. Organisation du Code Source

3.1. Arborescence

```
mediastock/
├── config/
│   ├── Database.php      # Classe PDO (ATTENTION: majuscule D - Linux sensible à la
│   │                     casse)
│   └── env.php           # return [...] avec credentials (PAS de define)
├── public/
│   ├── api/              # Points d'entrée API JSON
│   │   ├── gettoutItems.php # GET - liste tous les Items (Patrimoine informatique)
│   │   ├── additem.php      # POST - ajouter un Item
│   │   ├── updateitem.php   # POST - modifier un Item (+ archivage)
│   │   ├── addpret.php      # POST - Créer un prêt
│   │   ├── cloturepret.php  # POST - Restituer un prêt
│   │   └── findbyqrcode.php # GET - identifier un Item par QR Code
│   └── frontend/          # Interface utilisateur
│       ├── index.html      # Tableau de bord
│       ├── patrimoine.html # Patrimoine informatique (inventaire)
│       ├── scan.html       # Scanner QR / Créer & Restituer un prêt
│       ├── prets.html      # Liste des prêts actifs
│       └── assets/         # CSS, JS, images
├── src/
│   └── models/
│       ├── BaseModel.php   # Connexion PDO partagée
│       ├── Item.php        # Modèle ITEM
│       ├── Pret.php        # Modèle PRÊT
│       ├── Emprunteur.php  # Modèle EMPRUNTEUR
│       └── Administrateur.php # Modèle ADMIN (id, login, mot_de_passe_hash
│                               UNIQUEMENT)
├── sql/
│   └── init.sql             # Schéma BDD + données initiales (25 Items réels)
└── docker-compose.yml      # Définition des 3 services Docker
```

3.2. Conventions de Code

- PHP procédural avec PDO pour toutes les interactions BDD
- Séparation : logique métier dans /src/models, exposition dans /public/api
- Toutes les API retournent : Content-Type: application/json
- Format de réponse uniforme : { "success": bool, "message": "...", "data": ... }
- Gestion erreurs : try/catch PDOException avec retour JSON et code HTTP approprié
- CRITIQUE Linux : Database.php avec majuscule D (case-sensitive sur Linux)

4. Base de Données MySQL

4.1. Connexion

Paramètre	Valeur
Hôte (réseau Docker)	db
Port	3306
Base de données	mediastock
Utilisateur applicatif	mediastock_user / MediaStock06200@@&
Utilisateur root	root / MediaStock06200@@&

5. SCHÉMA

5.1. Schéma MCD

Le schéma ci-dessous représente le Modèle Conceptuel de Données de l'application MediaStock. Il a été conçu avec l'outil Looping et reflète fidèlement la structure de la base de données MySQL.

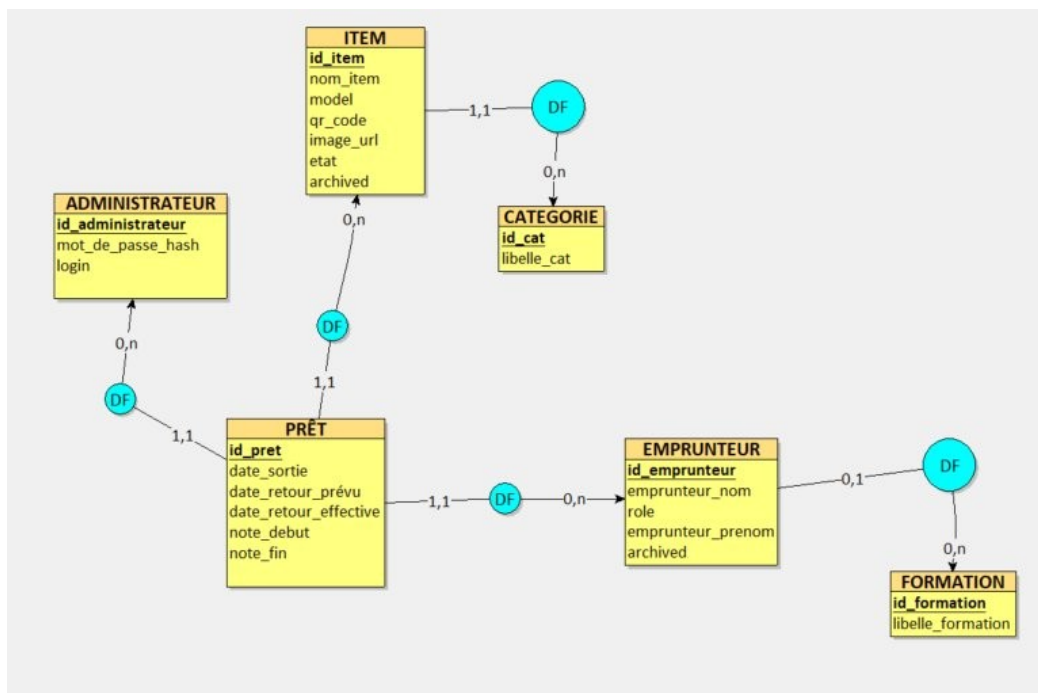


Figure 1 – Modèle Conceptuel de Données (MCD) MediaStock

5.2. Diagramme de cas (UML)

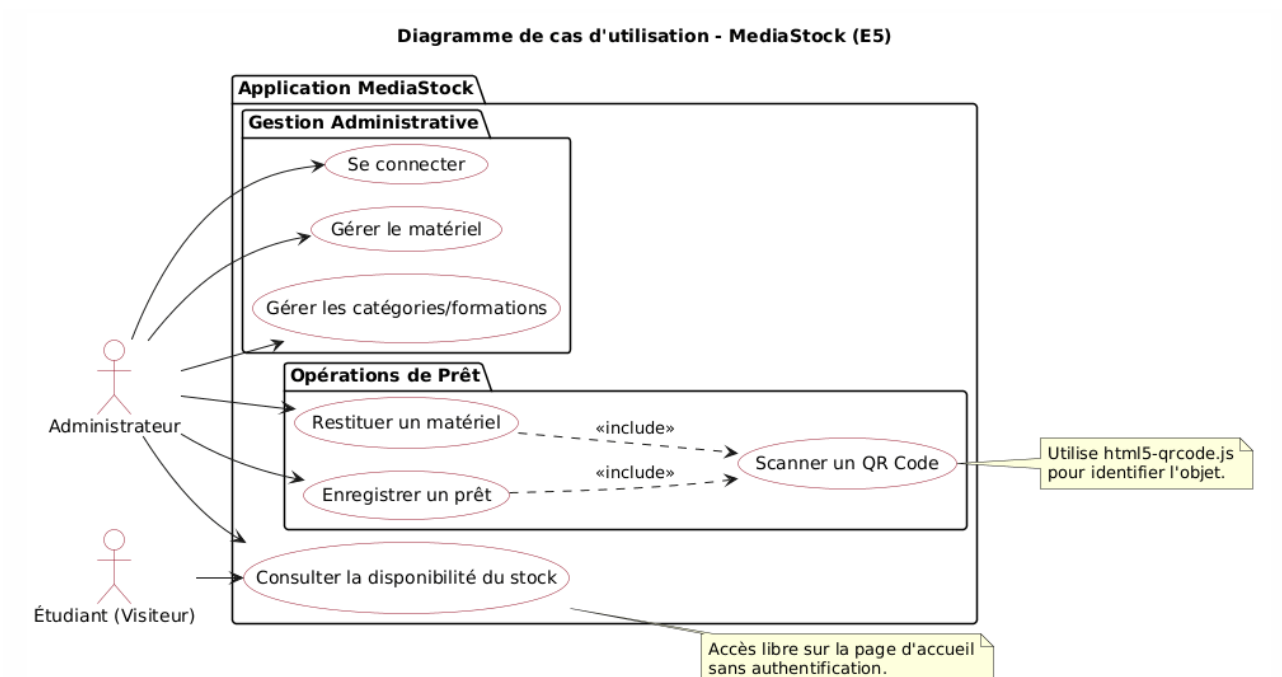


Figure 1 – MCD MediaStock (6 entités : ITEM, CATEGORIE, PRÊT, EMPRUNTEUR, FORMATION, ADMINISTRATEUR)

5. Documentation des API JSON

5.1. Convention Générale

Toutes les API sont dans /public/api/. Elles retournent du JSON avec Content-Type: application/json :

```
// Réponse succès
{ "success": true, "message": "Opération réussie", "data": { ... } }
// Réponse erreur
{ "success": false, "message": "Description erreur", "data": null }
```

5.2. gettoutItems.php – Liste du Patrimoine

Champ	Valeur
Méthode	GET
URL	/public/api/gettoutItems.php
Params	Aucun (retourne tous les Items, archived=0)
Réponse	{ success: true, data: [{ id_item, nom_item, model, qr_code, etat, libelle_cat, ... }] }

5.3. findbyqrcode.php – Identification par Scan QR

Champ	Valeur
Méthode	GET
URL	/public/api/findbyqrcode.php?qr={valeur_qr_code}
Params	qr (string) – valeur lue par html5-qrcode.js
Réponse	{ success: true, data: { id_item, nom_item, etat, prêt_actif: bool, id_pret } }
Utilisé par	L'interface scan.html pour identifier l'équipement avant prêt/retour

5.4. additem.php – Ajouter un Équipement

Champ	Valeur
Méthode	POST
URL	/public/api/additem.php
Body JSON	{ nom_item, model, qr_code, image_url, etat, id_cat }
Réponse	{ success: true, message: 'Item créé', data: { id_item: ... } }

5.5. updateitem.php – Modifier / Archiver un Équipement

Champ	Valeur
Méthode	POST
URL	/public/api/updateitem.php
Body JSON	{ id_item, nom_item, model, etat, id_cat, archived } – tous les champs à jour
Archivage	Passer archived: 1 pour désactiver logiquement l'équipement

5.6. addpret.php – Créer un Prêt

Champ	Valeur
Méthode	POST
URL	/public/api/addpret.php
Body JSON	{ id_item, id_emprunteur, id_administrateur, date_retour_prévu, note_debut }
Horodatage	date_sortie = NOW() côté serveur PHP (pas passé par le client)
Réponse	{ success: true, message: 'Prêt créé', data: { id_pret: ... } }

5.7. cloturepret.php – Restituer un Prêt

Champ	Valeur
Méthode	POST
URL	/public/api/cloturepret.php
Body JSON	{ id_pret, note_fin }
Horodatage	date_retour_effective = NOW() côté serveur PHP
Réponse	{ success: true, message: 'Prêt clôturé' }

5.8. Flux de Scan Complet – Exemple

Scénario : Scan QR Code → Création d'un prêt

```
// 1. L'utilisateur scanne le QR Code (html5-qrcode.js retourne la valeur)
const qrValue = 'PC-DELL-001';

// 2. Identifier l'équipement
const res = await fetch('/public/api/findbyqrcode.php?qr=' + qrValue);
const item = await res.json(); // { success: true, data: { id_item: 5, nom_item: '...', prêt_actif: false } }

// 3. Si disponible → Créer un prêt (bouton 'Créer un prêt')
const pret = await fetch('/public/api/addpret.php', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
```



```
body: JSON.stringify({ id_item: 5, id_emprunteur: 12, id_administrateur: 1,  
note_debut: 'RAS' })  
});
```

6. Modèles PHP – Couche Métier

6.1. BaseModel.php – Connexion PDO

Classe mère dont héritent tous les modèles. Lit env.php via return [...] :

```
class BaseModel {
    protected PDO $pdo;
    public function __construct() {
        $config = require __DIR__ . '/../../config/env.php'; // retourne un tableau
        $dsn = sprintf('mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4',
            $config['host'], $config['port'], $config['database']);
        $this->pdo = new PDO($dsn, $config['username'], $config['password'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    }
}
```

6.2. Méthodes des Modèles

Modèle	Méthode	Description
Item.php	getAll()	SELECT items non archivés + JOIN categorie
Item.php	findByQrCode(string \$qr)	Recherche par qr_code + info prêt actif
Item.php	create(array \$data)	INSERT avec validation champs requis
Item.php	update(int \$id, array \$data)	UPDATE tous champs + archivage (archived)
Pret.php	create(array \$data)	INSERT + date_sortie = NOW()
Pret.php	cloture(int \$id, string \$note)	UPDATE date_retour_effective + note_fin
Pret.php	getActifs()	SELECT prêts WHERE date_retour_effective IS NULL
Administrateur.php	findByLogin(string \$login)	SELECT login + mot_de_passe_hash UNIQUEMENT
Administrateur.php	verifyPassword(string \$pwd, string \$hash)	password_verify() PHP

7. Sécurité – Exigences Bloc 2 SLAM

7.1. Authentification Administrateur

Mécanisme	Implémentation
Identifiants	Login : admin / Mot de passe : MediaStock_06* (≠ 'password')
Stockage BDD	Colonne mot_de_passe_hash → password_hash(MediaStock_06*, PASSWORD_BCRYPT)
Vérification	password_verify(\$input, \$hash) – jamais de comparaison directe
Session PHP	session_start() + session_regenerate_id(true) après connexion réussie
Protection pages	Vérification \$_SESSION['admin'] sur chaque page admin
Déconnexion	session_destroy() + redirection login

7.2. Protection contre les Injections SQL

```
//  CORRECT - Requête préparée PDO
$stmt = $this->pdo->prepare('SELECT * FROM item WHERE qr_code = :qr');
$stmt->execute([':qr' => $qrCode]);

//  INTERDIT - Concaténation directe (vulnérable)
$sql = 'SELECT * FROM item WHERE qr_code = ' . $qrCode; // JAMAIS
```

7.3. Sécurité HTTPS / Traefik

- TLS obligatoire sur le port 4433 – tout HTTP est redirigé
- L'API de caméra html5-qrcode.js exige HTTPS (sécurité navigateur)
- MySQL non accessible hors du réseau Docker interne
- phpMyAdmin protégé par authentification manuelle (PMA_ARBITRARY=1)

7.4. Archivage Logique (intégrité des données)

Aucune suppression physique des Items ni des Emprunteurs. Le champ archived=1 désactive l'enregistrement tout en préservant l'historique des prêts associés :

```
-- Archivage d'un Item
UPDATE item SET archived = 1 WHERE id_item = :id;
-- Les prêts passés restent consultables dans l'historique
```

8. Déploiement et Exploitation

8.1. Procédure de Déploiement Initial

1. Cloner le dépôt Git sur le serveur hôte
2. Vérifier /config/env.php (return [...]) avec les bons credentials)
3. Lancer : `docker-compose up -d`
4. Attendre 15 secondes (initialisation MySQL) puis vérifier : `docker-compose logs db`
5. Initialiser la BDD : `docker-compose exec db mysql -u root -pMediaStock06200@@&mediastock < sql/init.sql`
6. Accéder à <https://mediastock.iris.a3n.fr:4433>
7. Se connecter : admin / MediaStock_06*

8.2. Commandes Docker Utiles

Commande	Description
<code>docker-compose up -d</code>	Démarrer tous les services
<code>docker-compose down</code>	Arrêter et supprimer les conteneurs
<code>docker-compose logs -f web</code>	Suivre les logs Apache/PHP
<code>docker-compose logs -f db</code>	Suivre les logs MySQL
<code>docker-compose exec db mysql -u root -p</code>	Accès MySQL CLI
<code>docker-compose restart web</code>	Redémarrer uniquement PHP/Apache

8.3. Diagnostic des Problèmes Fréquents

Symptôme	Solution
'Database.php not found'	Vérifier la majuscule : Database.php (pas database.php) – Linux sensible
'Call to a member function...'	env.php doit retourner <code>return [...]</code> et non <code>define()</code> . Vérifier la syntaxe.
Connexion MySQL échoue	Attendre 15s après up. Vérifier <code>MYSQL_HOST=db</code> dans l'env.
Caméra QR ne s'active pas	HTTPS requis. Vérifier certificat Traefik + autorisation caméra navigateur.
phpMyAdmin : accès refusé	Saisir manuellement : <code>serveur=db</code> , <code>user=root</code> , <code>pwd=MediaStock06200@@&</code>
API retourne HTML au lieu de JSON	Vérifier header('Content-Type: application/json') en tout début de fichier API.